

Subsystem Design Project Report

ENGR 111-K

Group KS5

Alex Santora, Dylan Iturbe, Alex Santora

12/11/25

I pledge my honor that I have abided by the Stevens Honor System

Introduction & Requirements

This project delivers an end to end IoT enabled smart watering system for indoor plant care. The system integrates a mechanical enclosure, an electrical sensing and actuation subsystem and embedded software with network connectivity. The goal is to reduce inconsistent manual watering by continuously measuring plant environment conditions and dispensing water using a repeatable control rule. Indoor plants often suffer from irregular watering because soil condition is not visible and environmental conditions change throughout the day. A small automated system is needed to measure key indicators and respond with watering without requiring the user to constantly check the plant. The system also needs a cloud data path so performance can be verified over multiple days.

Objectives:

Measure ambient temperature, relative humidity, soil moisture and ambient light using four sensing channels, to transmit telemetry to a cloud MQTT broker using a standardized topic structure for logging and plotting, to provide autonomous watering using a local control algorithm based on a soil moisture threshold and a daily watering requirement, to support multi day deployment with stable sampling, secure mounting and reliable electrical connectivity, and to enable repeatable assembly and maintenance through a printed enclosure that organizes components and protects wiring.

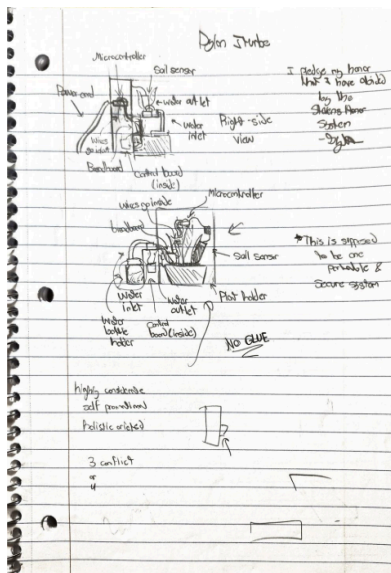
Functional/Software Requirements: Sampling all four sensors at a fixed interval of one set every 10 minutes, converting raw sensor outputs into engineering units on device, publishing to MQTT, and dispensing 20 mL of water once per day plus an additional 20 mL when soil moisture drops below a selected dry threshold in the recommended range.

Mechanical Requirements: Providing physical mounting and protection for the ESP32, sensors and actuator, supporting cable routing and strain relief so wiring does not loosen during deployment, and allowing access for sensor exposure, water delivery path alignment and maintenance.

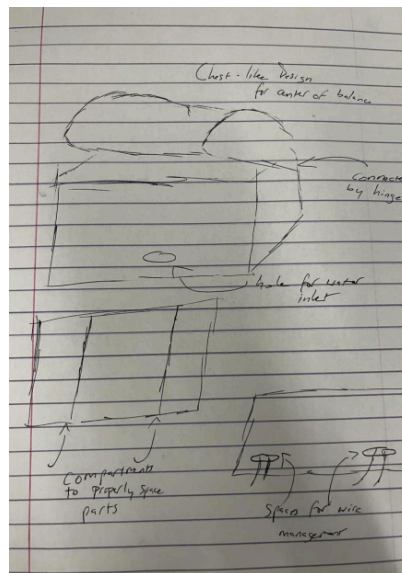
Electrical Requirements: Stable power delivery to the microcontroller and sensors, correct pin mapping for ADC, PWM and GPIO functions, and clean wiring practices with a defined color code and secure connections to reduce intermittent faults. These requirements define the system as a single integrated product because mechanical decisions affect sensor placement and water delivery accuracy, electrical decisions affect signal quality and software decisions affect sampling stability and network reliability.

Concept Design (Phase I & II)

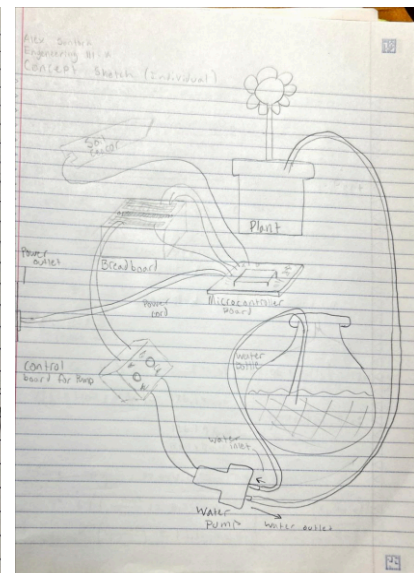
We began brainstorming ideas for the design of our project individually. With this, we were able to understand which types of designs each of us preferred. Eventually, once we were finished working on these designs, we shared them with each other.



(Dylan's Design)

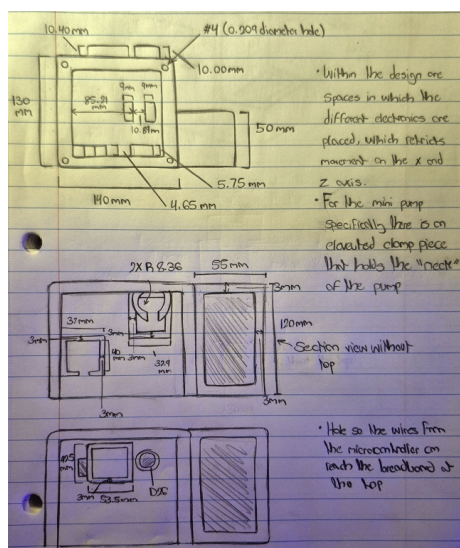


(Derek's Design)



(Alex's Design)

We reviewed each design carefully, taking into consideration the pros and cons of each one. Ultimately, we decided on Dylan's design, as it checked off all the requirements, such as being secure, transportable, and aesthetically pleasing. However, as the weeks progressed and we learned more about the restrictions placed on us, we decided it'd be best to revise the design instead. Here, we developed a design that is significantly more simplistic, but equally as

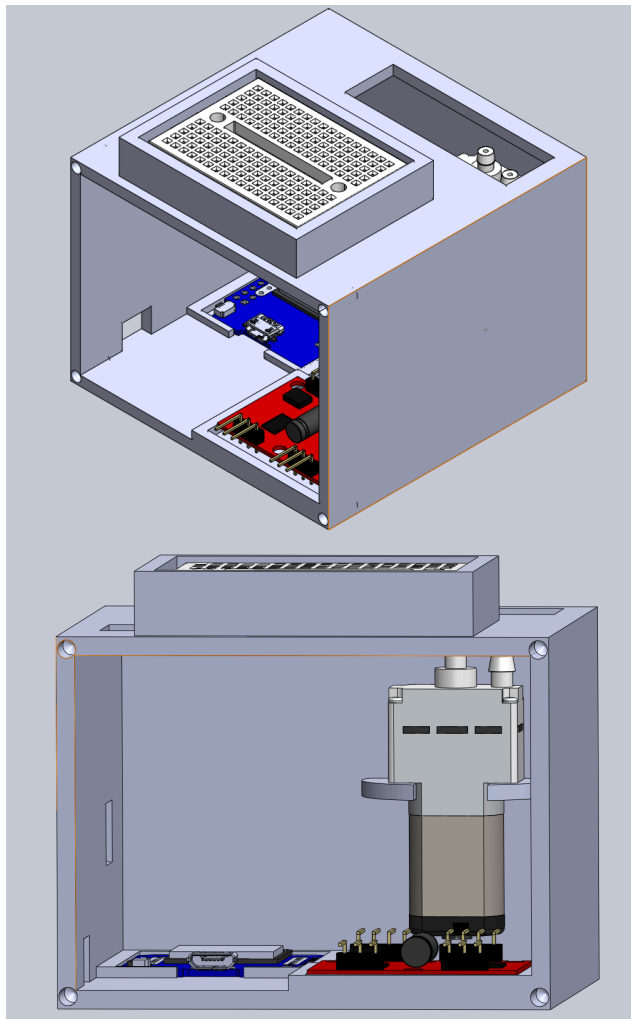
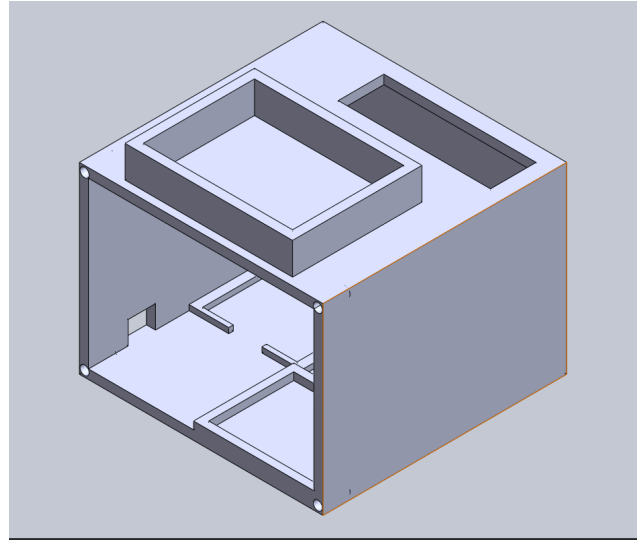


functional. It heavily reduces the complicated geometry found in the previous design, incorporates retaining walls to keep electronics in place, utilizes a screw-in door system to keep electronics safe, and a designated water holder so it all moves as one unit. While ambitious, we thought it'd be a perfect fit, and began 3D modeling it right away. Unfortunately, the design proved to be far too large, exceeding the guideline size limit and consequently, exceeding the printing time limit of eight hours. Yet again, we had to revise our design to abide by all constraints.

Unlike the first revision though, we decided to keep a large majority of the previous design. Instead, we made major changes to its dimensions, both for external and internal features through SolidWorks itself.

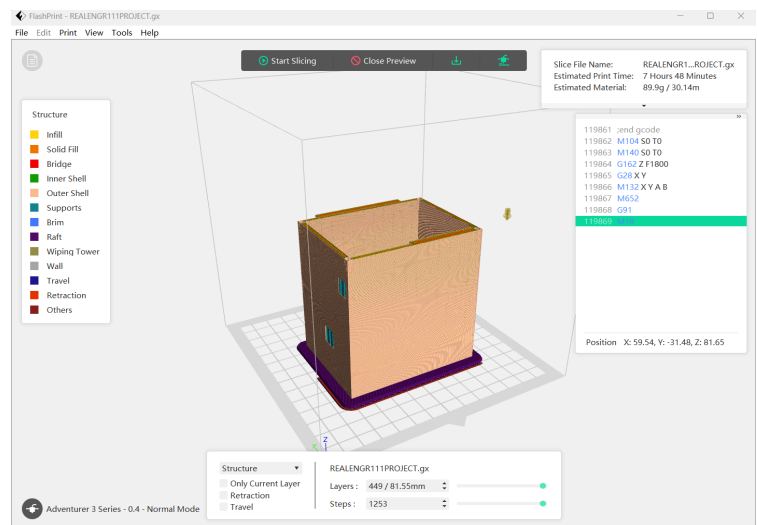
Mechanical Design

Ultimately, we finalized our design. It is considerably smaller than our predecessor, measuring only 60x80x80 mm, but maintains all functionality seen previously. As visible, there are slots where the several components fit perfectly, with ridges to prevent unexpected horizontal movement. Additionally, there are still purposeful holes where devices such as the water tubes and soil sensor can exit the system freely. However, there is no longer a designated “water bottle holder” as it requires too much material, and there is significantly less empty space.



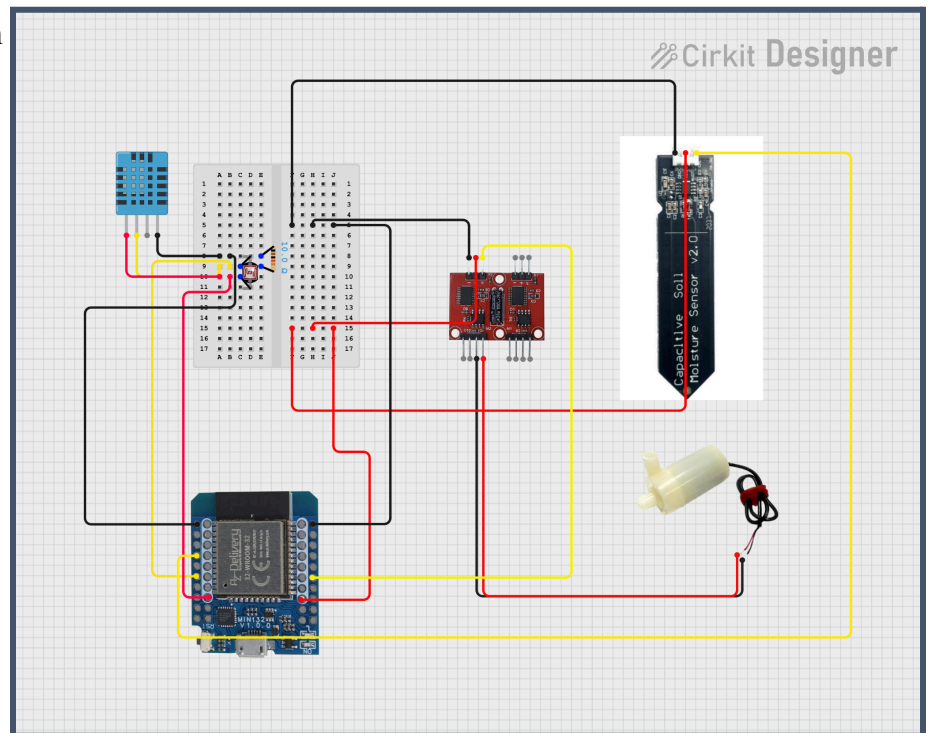
To simulate its real-world application with all other components, we made an assembly which introduced all relevant components in the SolidWorks Kit. As you can see, the MH-ET Live Mini Kit, FeeTech MiniPump Controller, Breadboard, and MiniPump are all held tightly and securely. Electricity would reach the MH-ET Live Mini Kit via USB-C which could be plugged in via the main opening or the small one to the left. Various wires would be used to then connect every component to one another, in an orderly and compact arrangement. If wires had to reach the Breadboard, there is a hole present to its left which allowed them to interact. While not shown in the screenshots, the soil sensor would remain outside but be connected via a small hole on the left side of the MH-ET Live Mini Kit. Similarly, the tubing necessary to

utilize the MiniPump would extend from the nozzles to the outside water bottle and plant via the hole at the top. Once we assured ourselves that the design was functional, we placed it in FlashPrint to view estimated time and material usage. In order to further reduce printing time as well as material usage, we oriented it vertically, with supports mostly used to maintain the small holes throughout the design. At 7hr 48min and 89.9g of material, our design is under the printing time limit, and thus is valid.



Electrical Design

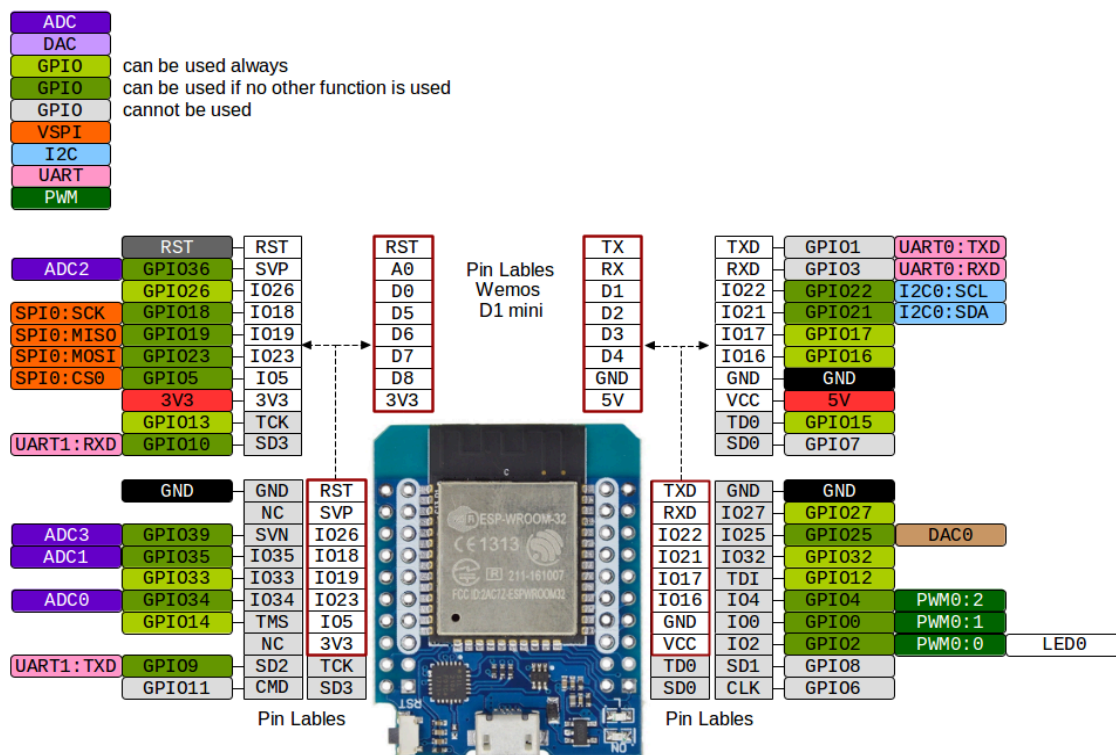
We decided upon the diagram shown to the right for our electrical design. Not only is it efficient, but it is color-coordinated and takes into account both the mechanical and software design components. Our color code for this project was simple: red wires were used for voltage or VCC pins, black wires were used for ground pins, and yellow wires were used for variable input/output pins. It is a simple yet effective way of limiting confusion and potential error in the system. One small design feature that could not be shown on this diagram, but could be seen in other photos included, is the connection between the DHT sensor and the breadboard. We decided to directly insert the sensor into the breadboard as opposed to connecting it with male to female jumper wires. Not only would this make the connection substantially more secure, but it also reduces the clutter of wires that could be present in these kinds of circuits.



To further the goal of efficient wire management, we also decided to provide direct connections between the variable wires of the mini pump and the soil moisture sensor to the microcontroller board. It was important that with the small housing of the system that the circuit would not be cramped or cluttered. The mechanical design allowed us to route the jumper wires through various openings of the shell. This meant that the wires could sit tightly and neatly within the design with a lower risk of them becoming disconnected from the breadboard or microcontroller board.;

Component	Pin #	Pin Type
Photocell	34	ADC
DHT Sensor	32	GPIO
Mini Pump	4	PWM
Soil Moisture Sensor	39	ADC

The table to the left shows the pin out numbers (corresponding to microcontroller pin diagram below) used for each component of the circuit. There were three pin types used in this circuit: ADC (analog-to-digital converter) for the photocell and soil moisture sensor, GPIO (general purpose input/output) for the DHT sensor, and PWM (Pulse Width Modulation) for the mini pump.



ADC, or analog-to-digital converter, is used to translate analog signals into digital data that computers can then process. In this case, light and soil moisture were not stored as digital values that could be understood by the program. Instead, the connection between these components and an ADC pin on the microcontroller board allowed the values to be converted to digital values.

The DHT sensor, however, was able to read temperature and humidity as a digital value, so it could be directly connected to a GPIO pin, or general purpose input/output. These types of pins send a direct digital signal to the microcontroller board, which means no conversion is needed. Lastly, our mini pump required a PWM signal due to its motor. Pulse width modulation allows the microcontroller board to directly control the analog-like signal of the motor digitally. This was thanks to components sorted out in our software design.

Software Design

The firmware is implemented as a single embedded application running on the ESP32 arduino board. It follows a cyclic executive model. Initialization happens in `setup()`. Continuous operation happens in `loop()`.

```
2025F_ENGR111_baseline.ino  SystemLogic.h

10
11 #include <DHT.h>
12 #include <ESP32Servo.h>
13 #include <WiFi.h>
14 #include <PubSubClient.h>
15 #include <WiFiClientSecure.h>
16 #include "SystemLogic.h"
17 #include "time.h"
18
19 // #define DHTTYPE DHT11 //define the type of DHT sensor
20 // DHT dht(32, DHTTYPE); //crate an instance of DHT sensor
21
22
23 // Global variable declarations or pin number settings can be done before the setup() function.
24 // Pin definitions
25 // Note! Pin number should match the actual PIN# used in your circuit
26 const int DHT_PIN = 32;
27 const int PHOTOCCELL_PIN = 34;
28 const int SOIL_PIN = 39; //adc
29 const int PUMP_PIN = 4;
30
31 //const int publishInterval = 100000; // Sampling time
32
33 // WiFi credentials
34 const char* WIFI_NAME = "Stevens-IoT";
35 const char* WIFI_PASS = "nMN882cmg7";
36
37 // MQTT configuration
38 const char* MQTT_SERVER = "98db5050a791439c98eac188febfece.s2.eu.hivemq.cloud";
39 const char* MQTT_USER = "stevens";
40 const char* MQTT_PASS = "Stevens@1870";
41 const int MQTT_PORT = 8883;
```

This block imports the libraries needed for sensing, actuation, networking and MQTT. It enables DHT temperature and humidity reads, servo based pump control, WiFi connectivity, secure client support and MQTT publishing. It also includes the `SystemLogic.h` helpers that handle startup and background tasks. It also defines the pin map for the DHT, soil sensor, photocell and pump

servo. It also stores the WiFi SSID and password plus the MQTT broker address, port and login. These constants centralize setup so wiring or network changes are easy to update.

```
42
43 // Team information
44 // You must follow the Topic naming protocol!
45 const String YEAR = "2025";
46 const String CLASS = "ENGR111";|
47 const String SECTION = "K";           // Use your section name
48 const String GROUP_NUMBER = "S5";     // Use your group number
49
50 // Sensor configuration
51 // Important! The values of below two variables should be updated with the values
52 //      measured by testing your Capacitive Soil Moisture sensor.
53 const int SOIL_MAXWET = 2210;
54 const int SOIL_MAXDRY = 3110;
55
56 // Variables for setting the operating range of the pump
57 const int PUMP_ON_ANGLE = 180;
58 const int PUMP_OFF_ANGLE= 90;
59
60 Servo minipump;
61
62 int watered=0;
63 int day=0;
64
65
66 void setup() {
67     wifi_start();
68
69     mqtt_start();
70     enable_sensors();
71     Serial.begin(115200);
72     pinMode(34,INPUT);
73     pinMode(32, INPUT); //set up pin mode for
74     pinMode(39, INPUT);
75     //begin to start sensor
76     minipump.attach(PUMP_PIN);
77 }
78
```

This block sets the identifiers used to build MQTT topics like year, class, section and group. It also defines soil sensor wet and dry calibration values so raw ADC counts can be converted into a percent. Finally, it defines the servo angles for pump on and off and creates the servo object and state tracking variables. The setup function runs once to bring the system online. It connects to WiFi, connects to MQTT, enables sensors, starts serial output, sets sensor pins as inputs and attaches the pump servo to its control pin. After this, the device is ready for the repeating control loop.


```

80
81 void loop() {
82   mqtt_loop();
83   // Measuring & Publishing Soil moisture sensor value: Start -----
84   int soilhumidity = analogRead(SOIL_PIN);
85   // Below commands convert(map) measured values to values between 0 and 100%
86   int soilPercent = map(soilhumidity, SOIL_MAXWET, SOIL_MAXDRY, 100, 0);
87
88   // Read Temperature in Celsius
89   int tempCel = dht.readTemperature();
90   int tempFar = tempCel*1.8 + 32; // Convert to Fahrenheit
91   // Read humidity as %
92   int Humidity = dht.readHumidity();
93
94   //Stores the information ouputed by the photocell in a float variable called x
95   float x = analogRead(34);
96   //converts analog to volts
97   float e = 2.718281828;
98   float z = 3.3*x/4095;
99   float y= pow(28.215*e,2.1544*z);
100  y=y*100;
101

```

This snippet is the core sampling cycle. It first services the MQTT client each loop to maintain connection health and keep the session alive. It then reads the soil moisture sensor using the ADC and converts the raw value into a percentage using the wet and dry calibration endpoints. The DHT sensor is polled for temperature and humidity and the temperature is converted from Celsius to Fahrenheit to match the required units for reporting. The photocell channel is also sampled via the ADC and then converted through a voltage calculation and a calibrated mathematical model to produce a value in lux that can be logged and graphed.

```

101
102   int count =0;
103   int watered1=0;
104
105   //waters plant if soil humidity is below 20 percent
106   if (soilPercent < 20){
107     minipump.write(180);
108     //leaves pump on to supply ~20ml
109     delay(11650);
110     count=count+1;
111     watered1=watered1+1;
112   }
113   minipump.write(90);
114
115   int time=millis();
116   int daytime=86400000;
117   int xyz=daytime*(day+1);
118
119   //waters plant once every 24 hours after starting
120   if(time>xyz){
121     minipump.write(180);
122     //leaves pump on to supply ~20ml
123     delay(11650);
124     watered1=watered1+1;
125     day++;
126   }
127

```

This snippet implements a rule based watering controller using two triggers. The first trigger is a soil moisture threshold rule that turns the pump on for a calibrated amount of time when soil moisture falls below 20 percent, which then it dispenses the fixed amount of water of approximately 20ml. The second trigger is a time based daily watering rule driven by milliseconds since boot, which forces one watering event per 24 hour period even if the soil does not cross the threshold. The controller updates runtime state variables so it can track whether watering occurred and it increments a day counter so the next daily watering window is scheduled correctly.

```
131
132     char pubString_soil_hum[8];    // Character array variable
133     // Below command Determines the format of the output data (the number of significant digits)
134     dtostrf(soilPercent, 1, 2, pubString_soil_hum); //dtostrf(float_value, min_width, num_digits_after_decimal, where_to_store_string)
135     String soilTopic = YEAR + "/" + CLASS + "/" + SECTION + "/" + GROUP_NUMBER + "/" + "soil_hum";
136
137     //Non-blocking MQTT publish & delay
138     client.publish(soilTopic.c_str(), pubString_soil_hum); // publish
139
140     char pubString_dht_hum[8];    // Character array variable
141     // Below command Determines the format of the output data (the number of significant digits)
142     dtostrf(Humidity, 1, 2, pubString_dht_hum); //dtostrf(float_value, min_width, num_digits_after_decimal, where_to_store_string)
143     String dhtHum = YEAR + "/" + CLASS + "/" + SECTION + "/" + GROUP_NUMBER + "/" + "Hum";
144
145     //Non-blocking MQTT publish & delay
146     client.publish(dhtHum.c_str(), pubString_dht_hum); // publish
147
148     char pubString_photocell[8];    // Character array variable
149     // Below command Determines the format of the output data (the number of significant digits)
150     dtostrf(y*100, 1, 2, pubString_photocell); //dtostrf(float_value, min_width, num_digits_after_decimal, where_to_store_string)
151     String Lux = YEAR + "/" + CLASS + "/" + SECTION + "/" + GROUP_NUMBER + "/" + "Lux";
152
153     //Non-blocking MQTT publish & delay
154     client.publish(Lux.c_str(), pubString_photocell); // publish
155
156
157
158
159
160     char pubString_dht_temp[8];    // Character array variable
161     // Below command Determines the format of the output data (the number of significant digits)
162     dtostrf(tempFar, 1, 2, pubString_dht_temp); //dtostrf(float_value, min_width, num_digits_after_decimal, where_to_store_string)
163     String dhtTemp = YEAR + "/" + CLASS + "/" + SECTION + "/" + GROUP_NUMBER + "/" + "Temp";
164
165     //Non-blocking MQTT publish & delay
166     client.publish(dhtTemp.c_str(), pubString_dht_temp); // publish
```

This snippet prepares sensor values for network transmission and publishes them to the broker. Each numeric measurement is converted into a compact text payload with consistent decimal formatting so subscribers can parse and graph values without ambiguity. Topics are assembled by concatenating the year, course, section and group identifiers with a sensor specific suffix. The MQTT client then publishes each measurement to its matching topic. This creates four independent telemetry streams, one per sensor type, which supports clean subscriptions and organized logging on the receiving side.

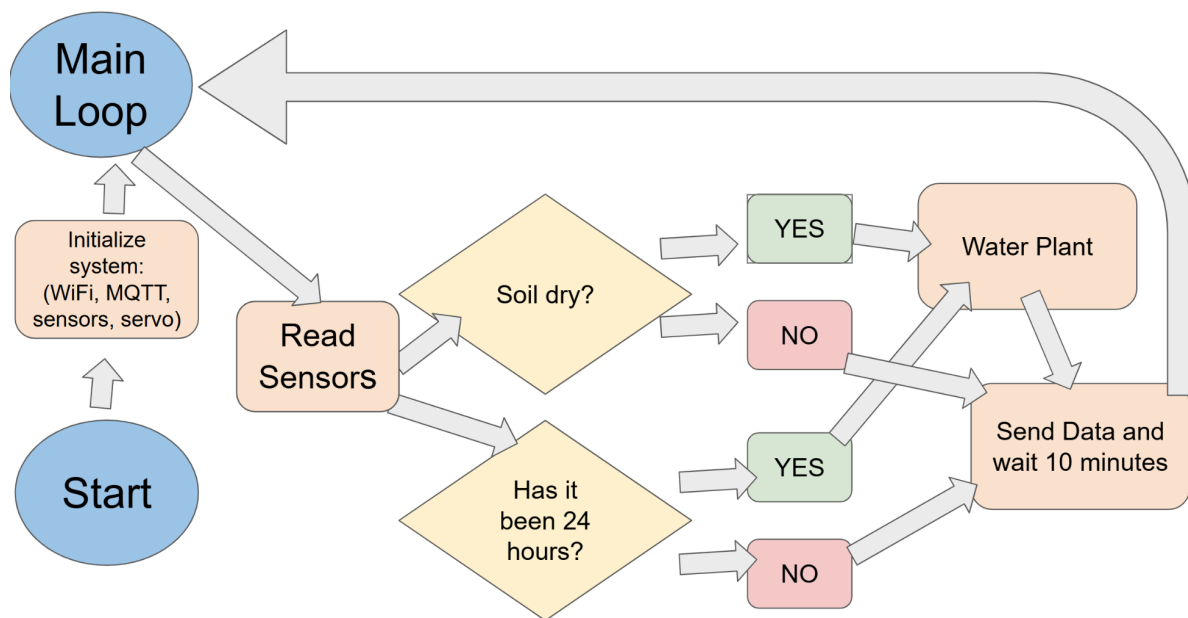
```

168
169 //ensures 10 minute delay even if the plant is watered
170 if(watered1==1){
171     delay(publishInterval-11650);
172 }
173 else{
174     delay(publishInterval);
175 }
176
177 }
178

```

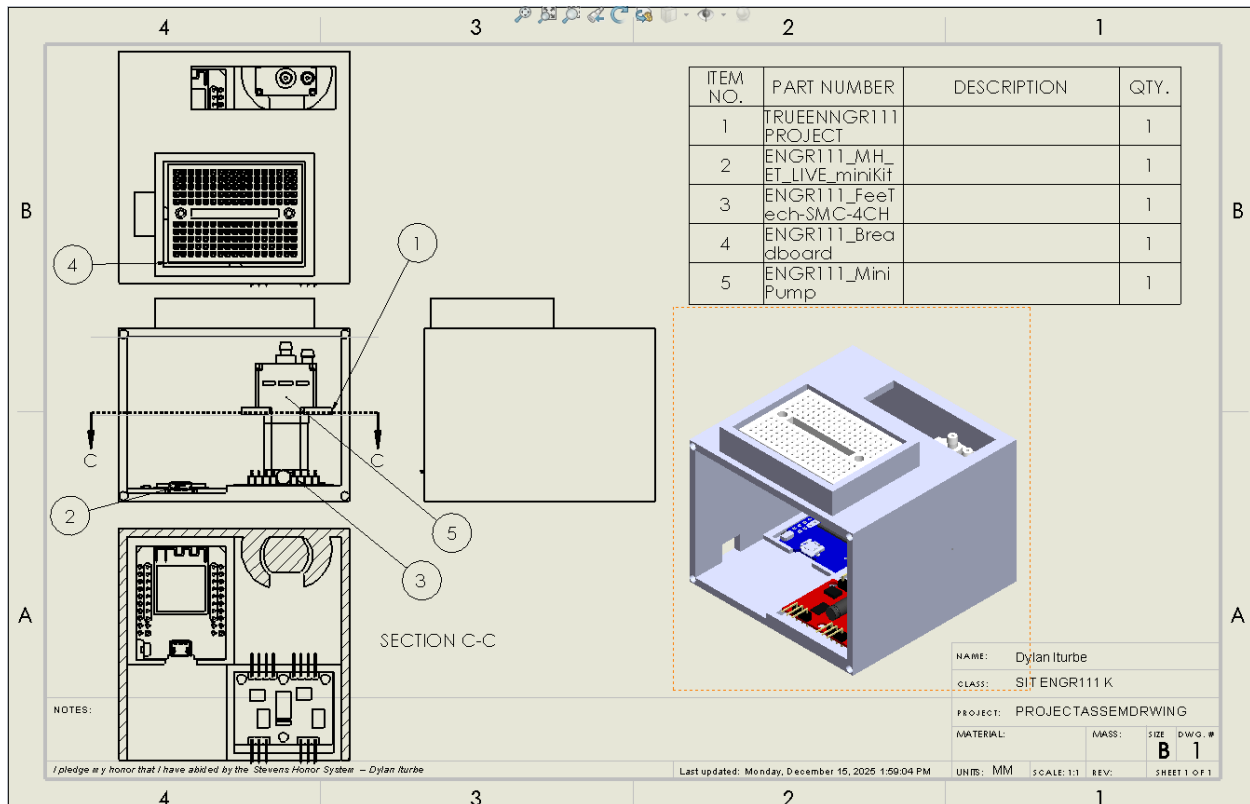
This block keeps the cycle period consistent. If watering happened, it subtracts the pump on time from the delay so the total loop time stays exactly the target interval. If no watering occurred, it waits the full sampling interval.

All this can be shown in an over simplified way through this flowchart:

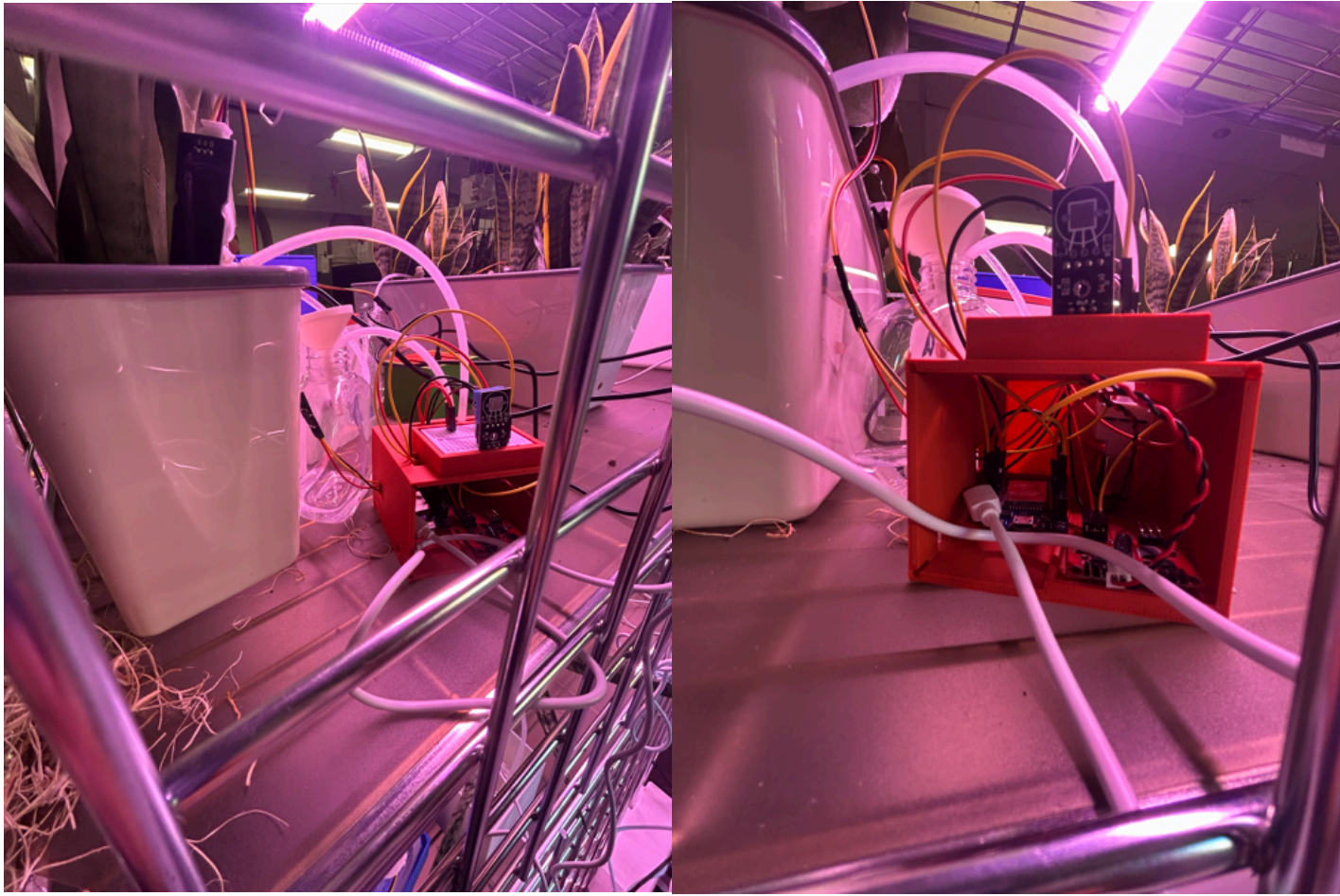


Assembly & Prototyping

Assembly for this design is rather simple, as it only consists of one part. However, in order to demonstrate its functionality, we constructed an assembly utilizing the SolidWorks Kit components we deemed necessary and converted it into a drawing with a list of materials.



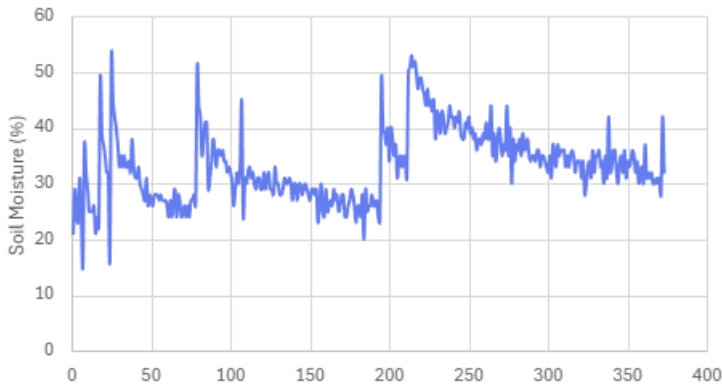
As seen, it consists of 5 main components, the main housing unit, and the 4 electrical devices used to make the automatic feeder function, omitting the DHT Sensor, as while it is essential, it is difficult to represent in the assembly. The balloons correspond to their numbered components and display how they would be oriented in the real-world counterpart. Additionally, here is the full design and system.



Design Testing and Evaluation

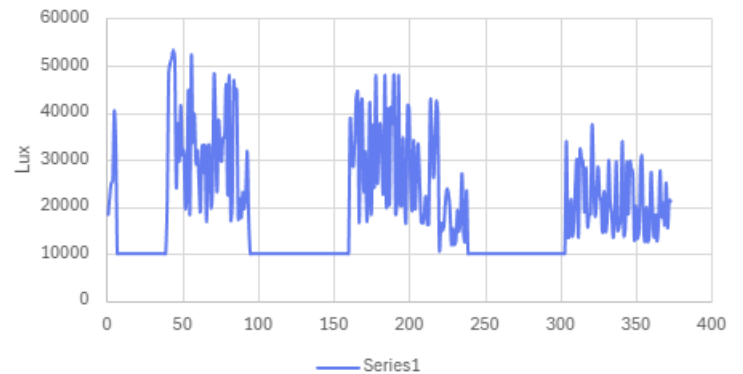
Our system went through hours of testing before it could finally be deployed. It took many different troubleshooting methods to bring down or bring up values that were being incorrectly displayed, or figure out why data could not be printed to the ENGR 111 subscriber website. Ultimately, by December 2nd at approximately 5:19 P.M, our system was functional, publishing data through the MQTT network. Below are our results from the period December 2nd through December 5th.

Soil Moisture: December 2nd- December 5th



For our lux values, they may be a bit off. We believe that there may have been an error somewhere in our code that sets the lux value off by a factor of 10. Taking that into account, the value would seem to be hovering from 1000 to about 4000-5000 lux.

Lux Values: December 2nd- December 5th



Air Humidity: December 2nd- December 5th



The air humidity seemed to have stayed relatively stable, with a couple of instances of the value dropping to 0. Though the values started slightly higher at 10-12 percent, the rest of the deployment period saw the value sitting at around 6-7%.

Lastly for the temperature, it mostly stayed in the range of 70-78 degrees fahrenheit. This was the temperature we expected, though there were two unexplained instances in which it dropped to 32 degrees fahrenheit. This could just be a temporary issue with the sensor that resolved itself.

Temperature: December 2nd- December 5th



Our deployment was initiated on December 2nd, 2025, and readings from the system ceased on December 5th, 2025. We set the data to be published on a ten minute interval, giving us a large sample size of data which was mostly stable, but had some outliers as well. Our system only ran for 4 days of collecting data because it was somehow unplugged at the start of December 6th and it went unnoticed.

Project Management

The project was managed by dividing the work into three integrated subsystems: mechanical design, electrical integration and embedded software. This structure allowed parallel development while keeping interface requirements clear, especially sensor placement, pin mapping and actuator control. Dylan led the mechanical work, including enclosure CAD and print planning. Derek led the electrical work, including circuit assembly, wiring organization and validation of signal integrity. Alex led the software work, including sensor acquisition, control logic and MQTT data publishing.

A team member agreement established at the start of the project defined roles, communication expectations and deadlines. The team used regular check-ins to track progress and address issues early. When additional integration time was needed, the group met outside of class to assemble the prototype, troubleshoot and verify that the full system operated correctly. As a result, tasks were completed on schedule and the final design reflects coordinated work across the mechanical, electrical and software components.

Conclusion

System performance was assessed through multi-day operation, during which soil moisture, temperature, humidity, and ambient light were sampled in fixed intervals and reliably transmitted to an MQTT broker. This data confirmed stable sampling, consistent network connectivity, and correct topic publishing; the ability for the system to autonomously water the plant when reaching its dry threshold and on a fixed time interval with the correct amount of water shows that our software requirements were met.

Our design also proved to be sturdy and reliable. Not only this, but it served to be compact, but not constricted. Our components sat tightly and securely in its housing, but were not on top of each other. Certain design features also allowed other parts of the design, such as the electrical design, to become much easier (i.e. openings in the mechanical design allowed proper cable management). We also did not experience any issues with our components being exposed to water, which is another requirement the design met.

Our electrical design was simple yet effective. Color coded wires prevented any sort of error from occurring, such as wires being connected to incorrect pins on the microcontroller board. Our use of different pin types such as ADC, GPIO, and PWM allowed us to convert analog signals that could not be interpreted by the microcontroller board into digital signals that it could then output.

The design we finished with was able to meet our three design requirements and objectives. Our goals for our mechanical, software, and electrical designs were achieved. Though we met most of our requirements and objectives, there are some improvements that could be made for a future design such as implementing a water bottle holder within the printing time requirement, or soldering wires to provide a more secure electrical connection. But, our main goal of an autonomous plant watering subsystem was achieved.